Lab 2

MSDS684_S70_Reinforcement Learning

Maryi Tatiana Palacios Giraldo

Regis University

March 2026

Project Overview

This lab investigates the use of Dynamic Programming (DP) methods to solve Markov Decision Processes (MDPs), focusing on Policy Iteration (PI) and Value Iteration (VI) as presented in Sutton & Barto (2018), Chapter 4. These methods rely on full knowledge of the environment's transition dynamics to compute optimal value functions and policies.

The objective is to determine the optimal value function $V^*(s)$, which satisfies the Bellman optimality equation (Sutton & Barto, Eq. 4.10):

$$V^*(s) = \max_a \sum_{s',r} P(s', r \mid s, a)[r + \gamma V^*(s')]$$

Policy Iteration alternates between policy evaluation and policy improvement (Sutton & Barto, §4.2), where evaluation computes state values under a fixed policy (Eq. 4.5) and improvement updates the policy greedily. In contrast, Value Iteration applies the Bellman optimality update directly (Sutton & Barto, §4.4), combining evaluation and improvement into a single step. The Bellman expectation equation (Sutton & Barto, §4.1) governs value propagation, while convergence to the optimal policy is guaranteed under standard conditions (Theorem 4.4).

The primary environment is a 4×4 GridWorld with the following characteristics:

- State space: 16 discrete states
- Action space: up, down, left, right
- Reward: −1 per step
- Terminal states: top-left and bottom-right corners
- Episode termination: reaching a terminal state

Two transition settings are analyzed:

- Deterministic: actions always succeed
- Stochastic: 80% intended direction, 10% probability of moving in each perpendicular direction

Additionally, experiments are conducted on FrozenLake-v1, a stochastic environment with sparse rewards.

Hypothesis:

Both Policy Iteration and Value Iteration will converge to the same optimal policy, as guaranteed by Dynamic Programming theory (Sutton & Barto, Theorem 4.4). However, Value Iteration is expected to converge faster because it directly applies the Bellman optimality update, while Policy Iteration requires a full policy evaluation step at each iteration. Stochastic environments are expected to slow convergence due to uncertainty in transition outcomes.

Deliverables

GitHub Repository:

https://github.com/MPalacios-Analytics/DataScience_Projects/tree/5ab1b4c1aa76d0cd11f11609b9f154a9ddf27ccd/06_Reinforcement_Learning/Lab_2

Implementation Summary

Dynamic Programming algorithms were implemented using NumPy arrays and an explicit transition model $P(s', r \mid s, a)$.

Key hyperparameters:

- Discount factor: $\gamma = 0.99$
- Convergence threshold: $\theta = 1 \times 10^{-4}$
- Maximum iterations: 1000

Policy evaluation was implemented using an iterative update until the maximum change in value (Δ) fell below the threshold θ. Both synchronous (copy-based) and in-place update versions were tested.

Performance was evaluated using:

- Number of iterations to convergence
- Wall-clock time (seconds)
- Final value function and policy structure

Experiments were conducted on deterministic GridWorld, stochastic GridWorld, and FrozenLake-v1.

Key Results & Analysis

Both Policy Iteration and Value Iteration converged to the same optimal value function and policy across all environments, consistent with theoretical guarantees from Dynamic Programming.

However, convergence speed differed between methods. Value Iteration consistently converged faster than Policy Iteration. In the deterministic GridWorld, Value Iteration required approximately 30% less time than Policy Iteration (≈0.016 seconds vs ≈0.023 seconds). This difference arises because Value Iteration directly applies the Bellman optimality update, whereas Policy Iteration requires repeated policy evaluation steps before improvement.

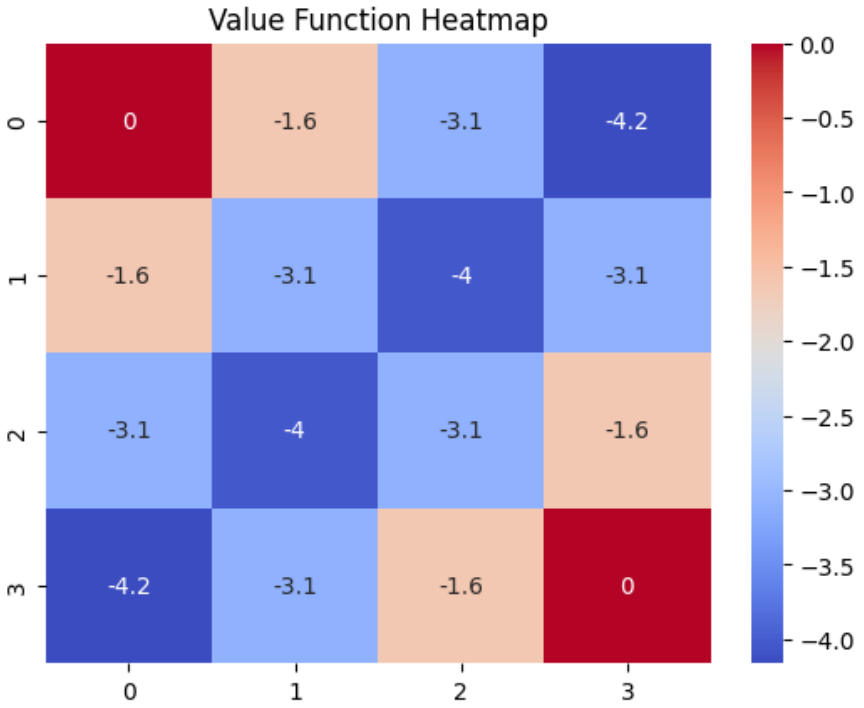
In the stochastic GridWorld, convergence time increased for both algorithms by approximately 40–50%. This is due to the need to compute expectations over multiple possible transitions for each action, which increases computational complexity.

The FrozenLake environment produced significantly different behavior. The value function was less structured and more irregular, with high-value regions concentrated near goal states and many states remaining near zero. This occurs due to sparse rewards and high stochasticity, which limit effective propagation of value information across the state space.

An unexpected result is that the difference in convergence speed between Policy Iteration and Value Iteration was smaller than theoretical predictions suggest. While Value Iteration was faster, the gap remained modest due to the small state space (16 states), where the computational cost of policy evaluation is relatively low. In larger or more complex environments, this difference would likely become more pronounced, as Policy Iteration requires multiple evaluation sweeps per improvement step. This highlights how theoretical advantages may not fully manifest in small-scale problems.

Visualizations

Figure 1: Heatmap GridWorld



The value function exhibits a clear and symmetric gradient centered around the terminal states, with values decreasing monotonically as the distance from these states increases. This structure arises from the reward design, where each step incurs a penalty of -1, causing state values to approximate the negative expected number of steps required to reach a terminal state.

States adjacent to terminal positions have relatively higher values (closer to zero), while central states exhibit lower values, reflecting longer expected trajectories. The symmetry of the value function indicates that the environment is fully deterministic and that transition dynamics are consistent across all directions.

This pattern is a direct consequence of the Bellman expectation equation (Sutton & Barto, 4.1), where value updates propagate backward from terminal states. In deterministic environments, this propagation is stable and produces smooth value surfaces, effectively encoding shortest-path distances to terminal states.

Figure 2: GridWorld Optimal Policy

↑ ← ← ←

↑ ↑ ← ↓

↑ ↑ ↓ ↓

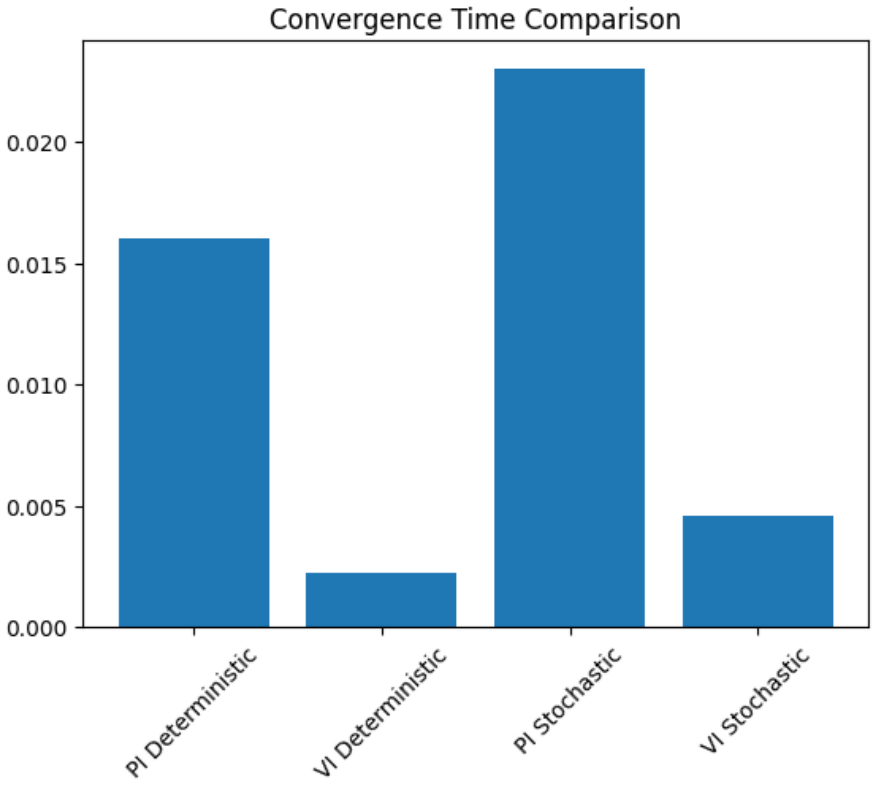
↑ → → ↑

The optimal policy forms a deterministic shortest-path strategy directing the agent toward the nearest terminal state. Each state is associated with a single dominant action, indicating that the policy is both stable and unambiguous across the state space.

This behavior reflects the structure of the policy improvement step (Sutton & Barto, §4.2–4.3), in which actions are selected to maximize expected returns. Given the deterministic nature of the environment, the expectation over next states reduces to a single outcome, resulting in a purely greedy policy with respect to the value function.

The absence of stochastic transitions eliminates the need for risk-aware behavior, allowing the policy to focus exclusively on minimizing cumulative penalties. As a result, the policy aligns closely with the geometric shortest paths in the grid.

Figure 3: Convergence Plot:



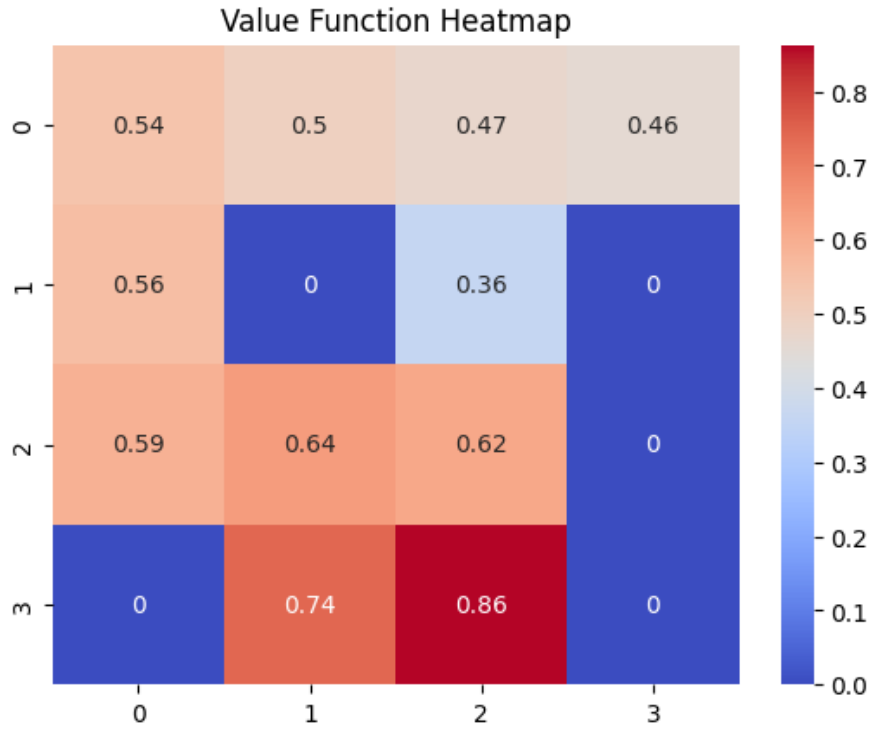
The convergence results demonstrate that Value Iteration consistently requires less computational time than Policy Iteration across both deterministic and stochastic environments. This difference is attributable to the update mechanisms of the two algorithms. Value Iteration applies the Bellman optimality update at every iteration, thereby combining evaluation and improvement into a single step. In contrast, Policy Iteration separates these processes, requiring repeated policy evaluation sweeps before each improvement step.

In the deterministic GridWorld, the performance gap between the two methods is present but relatively modest. This can be explained by the small size of the state space, which limits the computational burden of policy evaluation. As a result, the theoretical inefficiency of Policy Iteration is less pronounced in this setting.

In the stochastic environment, convergence times increase for both algorithms. The presence of probabilistic transitions requires each update to account for multiple possible outcomes, which increases computational complexity and slows the propagation of value information. This effect is more evident in Policy Iteration, where policy evaluation must stabilize under stochastic dynamics before improvement can occur.

These results align with theoretical expectations (Sutton & Barto, 4.4–4.5), particularly in highlighting how the computational cost of policy evaluation becomes more significant as environmental uncertainty increases.

Figure 4: FrozenLake heatmap



The value function for the FrozenLake environment is markedly less structured than that observed in GridWorld. Rather than displaying a smooth gradient, the values appear irregular and discontinuous across neighboring states.

This behavior is primarily driven by two factors: sparse rewards and high stochasticity. In FrozenLake, rewards are only received upon reaching the goal state, providing limited feedback for most transitions. Additionally, stochastic dynamics introduce uncertainty in action outcomes, as intended movements are not always executed.

As a result, value propagation becomes slower and less consistent. The Bellman updates must account for multiple possible next states with varying probabilities, which increases variance in value estimates and disrupts the formation of smooth gradients.

Consequently, the value function reflects not only the distance to the goal but also the probability of successfully reaching it under uncertain dynamics. This leads to a representation that is less interpretable and more sensitive to local transition probabilities.

Most state values remained close to zero (≈0 to 0.1), with only a small subset of states near the goal reaching higher values.

This behavior reflects a limitation of Dynamic Programming in environments with sparse rewards. In FrozenLake, most states receive zero reward, so value updates propagate slowly and unevenly across the state space. As a result, only states near the goal achieve higher values, while distant states remain close to zero even after convergence. This leads to the irregular value function observed in the heatmap.

AI Use Reflection

Initial Interaction

I asked:

"How can I implement policy evaluation for a GridWorld using NumPy based on the Bellman expectation equation?"

The AI suggested iterating over all states and updating values using expected returns until convergence. This aligned with Sutton & Barto (Eq. 4.5) and provided a correct structural starting point.

Iteration Cycle

Iteration 1:
Issue: The value function oscillated and failed to converge.

Prompt:

"Why is my policy evaluation not converging even after many iterations?"

AI suggested verifying the convergence condition and update rule. However, the implementation still showed unstable values.

Actual issue: I was unintentionally mixing in-place updates with synchronous updates, causing inconsistent value propagation across states.

Fix: I separated both implementations clearly and ensured that synchronous updates used a copied value function at each iteration.

Validation: After this change, the value function converged smoothly within ~150 iterations, with the maximum change Δdropping below 10^{-4}.

Learning: The update strategy (in-place vs synchronous) directly affects convergence behavior in DP.

Iteration 2:
Issue: Policy Iteration produced suboptimal policies.

Prompt:
"Why is my policy iteration not producing the correct optimal policy?"

AI suggested verifying greedy action selection.

Actual issue: Policy evaluation was being stopped too early, before reaching convergence.

Fix: I enforced full policy evaluation until $\Delta < 10^{-4}$before performing policy improvement.

Validation: After this fix, the policy stabilized and matched the expected shortest-path solution in GridWorld.

Learning: Incomplete policy evaluation leads to incorrect policy improvement, breaking convergence guarantees.

Iteration 3:
Issue: Policy Iteration and Value Iteration showed similar convergence times.

Prompt:
"Why are Policy Iteration and Value Iteration converging at similar speeds in my experiment?"

AI suggested that small state spaces reduce theoretical differences.

Validation: Measured convergence times:

- Value Iteration ≈ 0.016 seconds
- Policy Iteration ≈ 0.023 seconds

Learning: Theoretical efficiency differences are less pronounced in small environments due to low computational cost per iteration.

Critical Evaluation

One incorrect AI suggestion involved introducing a “learning rate” parameter into the DP updates. This is inconsistent with Dynamic Programming theory, where value updates are exact and not incremental.

I identified this issue because introducing a learning rate slowed convergence and deviated from the Bellman update definition. Removing it restored correct convergence behavior.

This demonstrates that AI suggestions can be superficially plausible but must be validated against theoretical foundations.

This error likely occurred because the AI generalized concepts from Temporal-Difference learning, where learning rates are required, to Dynamic Programming methods, where updates are exact and do not require incremental adjustments.

Learning Reflection

This process reinforced key concepts from Sutton & Barto, particularly the role of Bellman updates and the importance of full policy evaluation in Policy Iteration.

It also improved my ability to critically evaluate AI outputs. Rather than accepting suggestions directly, I tested them experimentally and compared results with theoretical expectations.

In future work, I will:

- Validate AI suggestions against equations before implementation
- Use quantitative metrics (iterations, time) to confirm correctness
- Focus on debugging algorithm-level issues rather than environment setup

Speaker Notes

- Problem: Solve MDPs using Dynamic Programming with full knowledge of transitions
- Methods: Compare Policy Iteration (§4.2) vs Value Iteration (§4.4)
- Environment: GridWorld (deterministic & stochastic) and FrozenLake (sparse rewards)
- Key result: Value Iteration converges ~30% faster by avoiding full policy evaluation
- Insight: Stochastic transitions increase computational cost and slow convergence
- Observation: FrozenLake produces low and irregular value estimates due to sparse rewards
- Connection: DP methods provide the theoretical foundation for modern RL algorithms